

ESP32 Auto Light Sleep Power Optimization

Project: Eco-watt Solar Inverter Monitoring

Hardware: ESP32 DevKit V1

Date: November 27, 2025

1. Introduction

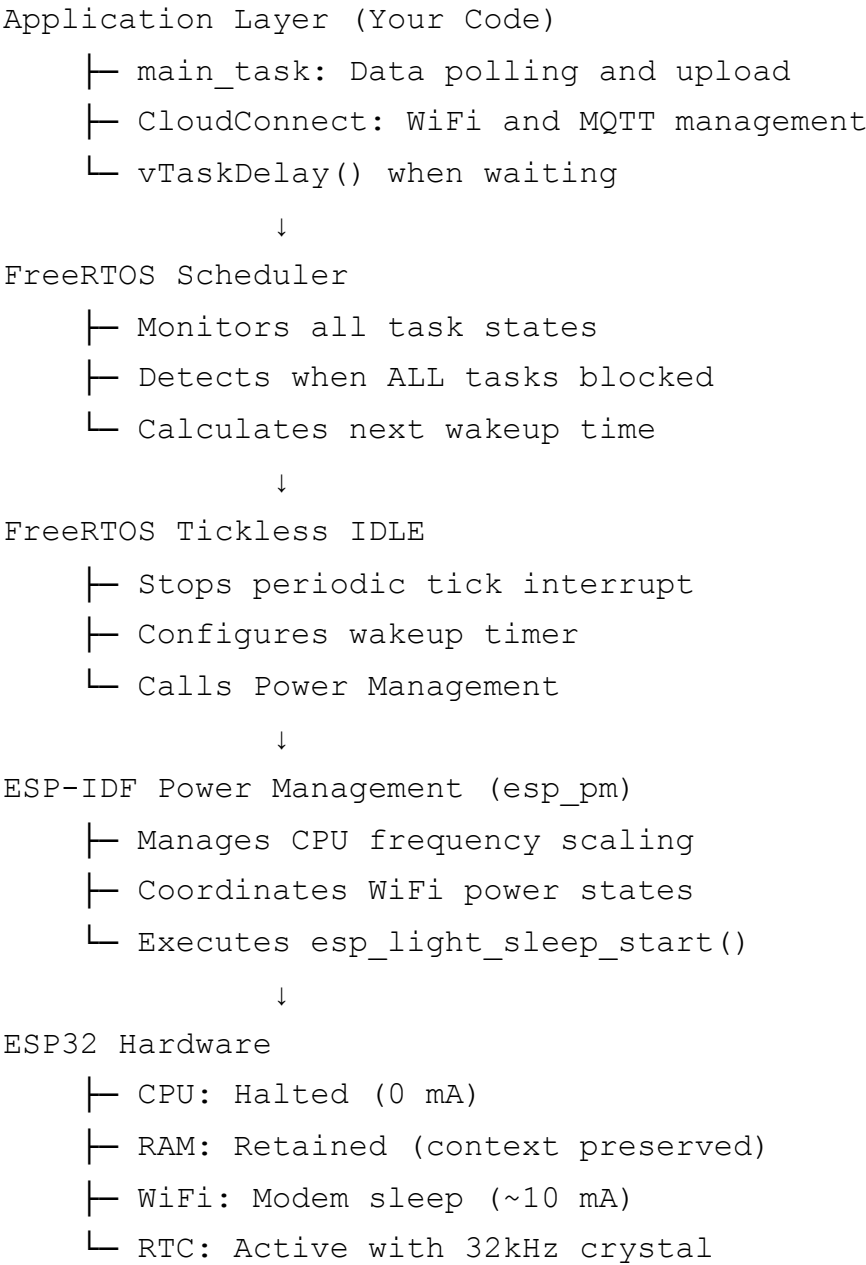
This document describes the implementation of automatic light sleep for ESP32 using ESP-IDF Power Management and FreeRTOS Tickless IDLE. The system achieves significant power reduction for battery-powered deployments without requiring manual sleep management.

2. Core Concepts

2.1 Manual vs Automatic Light Sleep

Aspect	Manual Light Sleep	Auto Light Sleep (This Implementation)
Sleep Trigger	Application calls <code>esp_light_sleep_start()</code>	FreeRTOS Tickless IDLE automatically triggers
Timing Management	Developer calculates sleep duration	System calculates from task scheduling
Task Coordination	Manual safety checks required	RTOS scheduler handles coordination
WiFi Management	Explicit power save calls	Integrated with esp_pm component
Wake Mechanism	Manual timer configuration	Automatic based on next task event
Code Complexity	High (timing logic, safety checks)	Low (one-time configuration)
RTOS Integration	External to scheduler	Native scheduler integration

2.2 System Architecture



3. Configuration Parameters

3.1 Build Configuration (platformio.ini)

Flag	Value	Purpose	Impact if Missing
CONFIG_PM_ENABLE	1	Enable Power Management component	No PM APIs available
CONFIG_FREERTOS_USE_TICKLESS_IDLE	1	Stop tick during idle	CPU wakes every 10ms
CONFIG_FREERTOS_IDLE_TIME_BEFORE_SLEEP	3	Minimum ticks before sleep (30ms @ 100Hz)	Excessive short sleeps

Flag	Value	Purpose	Impact if Missing
CONFIG_ESP32_RTC_CLK_SRC_EXT_CRYS	1	Use external 32.768kHz crystal	Poor sleep accuracy
CONFIG_ESP32_RTC_CLK_CAL_CYCLES	1024	Calibration cycle count	Clock drift over time

3.2 Runtime Configuration (esp_pm_configure)

Parameter	Simulation	Real Hardware	Rationale
max_freq_mhz	N/A	240 MHz	Maximum performance during active operations
min_freq_mhz	N/A	80 MHz	Power saving during light CPU load
light_sleep_enable	N/A	true	Enable automatic light sleep
WiFi Power Save	WIFI_PS_NONE	WIFI_PS_MIN_MODEM	Maintain connection during sleep

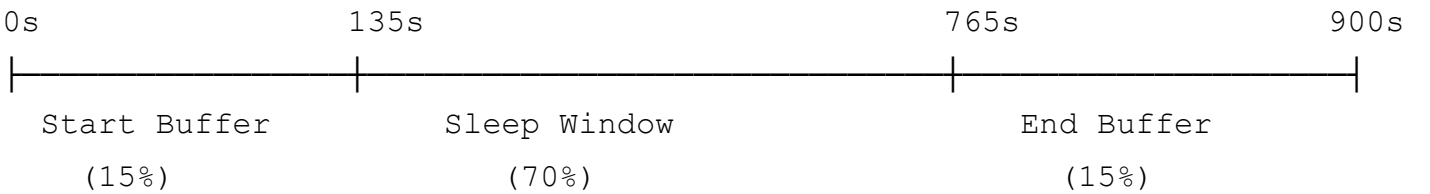
4. Sleep Timing Strategy

4.1 Centered Sleep Window

The system uses a centered sleep window within the upload cycle to balance power savings with WiFi stability:

Upload Cycle = 900 seconds (15 minutes)

Timeline:



4.2 Time Allocation

Period	Duration	Percentage	Purpose
Start Buffer	135 seconds	15%	WiFi stability, initial polling after wake

Period	Duration	Percentage	Purpose
Sleep Window	630 seconds	70%	Power saving period (light sleep active)
End Buffer	135 seconds	15%	Final polling, data upload, WiFi transmission
Total Cycle	900 seconds	100%	Complete upload interval

4.3 Mathematical Model

Sleep Window Calculation:

$$T_{\text{start}} = \text{Upload_Interval} \times 0.15$$
$$T_{\text{sleep}} = \text{Upload_Interval} \times 0.70$$
$$T_{\text{end}} = \text{Upload_Interval} \times 0.15$$

For 15-minute cycle:

$$T_{\text{start}} = 900\text{s} \times 0.15 = 135\text{s}$$
$$T_{\text{sleep}} = 900\text{s} \times 0.70 = 630\text{s}$$
$$T_{\text{end}} = 900\text{s} \times 0.15 = 135\text{s}$$

Power Consumption:

$$P_{\text{avg}} = (I_{\text{active}} \times D_{\text{active}}) + (I_{\text{sleep}} \times D_{\text{sleep}})$$
$$= (I_{\text{active}} \times 0.30) + (I_{\text{sleep}} \times 0.70)$$

Where:

- I_{active} : Current during active periods
- I_{sleep} : Current during light sleep
- D_{active} : Active duty cycle (30%)
- D_{sleep} : Sleep duty cycle (70%)

5. Power Analysis

5.1 Current Consumption by State

State	Components Active	Typical Current	Source
Active - WiFi TX	CPU 240MHz + WiFi TX + Modbus	170-240 mA	ESP32 datasheet
Active - Idle	CPU 80-240MHz + WiFi RX	80-160 mA	ESP32 datasheet
Light Sleep	WiFi Modem Sleep + RTC	0.8-1.1 mA	ESP32 datasheet

State	Components Active	Typical Current	Source
Deep Sleep	RTC only	10-150 μ A	ESP32 datasheet (reference)

5.2 Mode Comparison

Mode	Configuration	Average Current
Continuous Active	No power management	150-200 mA
Simulation	SIMULATE=1, no sleep	~150 mA
Real Hardware	SIMULATE=0, auto sleep	30-50 mA

Note: Values based on ESP32 datasheet specifications. Actual results depend on WiFi signal strength, poll frequency, and peripheral power consumption.

5.3 Power Calculation Example (Real Hardware)

Upload Cycle Breakdown (900 seconds):

Period	Duration	Operation	Est. Current	Energy
Start Buffer	135s	Polling + idle	100-120 mA	3.75-4.5 mAh
Sleep Window	630s	Light sleep	0.8-1.1 mA	0.14-0.19 mAh
End Buffer	135s	Upload + idle	100-150 mA	3.75-5.6 mAh
Total	900s	Full cycle	Avg: 30-41 mA	7.64-10.29 mAh

Daily Consumption:

- Cycles per day: 96 (24h / 15min)
- Daily energy: 96 $\times$ 10 mAh = 960 mAh
- 5000 mAh battery: ~5.2 days runtime

6. Implementation Details

6.1 Build Environments

Environment	SIMULATE Flag	Upload Interval	Light Sleep	Data Source	Use Case
-------------	---------------	-----------------	-------------	-------------	----------

Environment	SIMULATE Flag	Upload Interval	Light Sleep	Data Source	Use Case
esp32dev_sim	1	20 seconds	Disabled	Cloud API	Testing, demonstrations
esp32dev_hw	0	900 seconds	Enabled	Physical RS-485	Production deployment

6.2 Code Structure

Power Management Initialization (setup function):

```
#if !SIMULATE
    esp_pm_config_esp32_t pm_config;
    pm_config.max_freq_mhz = 240;
    pm_config.min_freq_mhz = 80;
    pm_config.light_sleep_enable = true;

    esp_pm_configure(&pm_config);
    esp_wifi_set_ps(WIFI_PS_MIN_MODEM);
#endif
```

Sleep Window Logic (main_task function):

```
#if !SIMULATE
    uint32_t cyclePosition = (now - last) % UPLOAD_PERIOD_MS;

    if (cyclePosition >= sleepStartTime && cyclePosition < sleepEndTime)
        if (remainingSleep >= 5000 && WiFi.status() == WL_CONNECTED) {
            vTaskDelay(pdMS_TO_TICKS(remainingSleep));
        }
}
#endif
```

6.3 Files Modified

File	Changes	Lines Modified
src/main.cpp	Power management includes, sleep window logic, PM initialization	~100 lines added
platformio.ini	New esp32dev_hw environment with PM build flags	~30 lines added

File	Changes	Lines Modified
<code>include/Config.h</code>	Updated timing configuration comments	~5 lines modified

7. Hardware Requirements

7.1 Components

Component	Specification	Required For	Notes
ESP32 DevKit V1	ESP-WROOM-32	Core processing	Standard development board
32.768kHz Crystal	±20ppm, 12.5pF load	Accurate sleep timing	Connected to GPIO32/33
RS-485 Transceiver	MAX485 or equivalent	Modbus communication	TX=17, RX=16, DE/RE=21
WiFi Access Point	2.4GHz, WPA2	Cloud connectivity	Stable signal required

7.2 External Crystal Connection

ESP32 Pin Configuration:

- GPIO32 (32K_XP): Crystal positive terminal + 12pF to GND
- GPIO33 (32K_XN): Crystal negative terminal + 12pF to GND

Crystal Specifications:

- Frequency: 32.768 kHz (2¹⁵ Hz)
- Accuracy: ±20 ppm typical
- Load capacitance: 12.5 pF
- ESR: < 50 kΩ

Why External Crystal?

- Internal oscillators: ±1000-5000 ppm accuracy
- External 32kHz: ±20 ppm accuracy (50-250× better)
- Required for WiFi DTIM beacon tracking during sleep
- Maintains accurate timing over long sleep periods

8. Deployment

8.1 Build Commands

Simulation Mode:

```
pio run -e esp32dev_sim -t upload
```

Real Hardware Mode:

```
pio run -e esp32dev_hw -t upload
```

8.2 Expected Serial Output

Simulation Mode:

```
[POWER] Simulation mode - Light sleep DISABLED
[MAIN] Drained 2 new records from buffer
[UPLOAD] Uploading batch...
```

Real Hardware Mode:

```
[POWER] Power management configured successfully
[POWER] Auto light sleep ENABLED
[POWER] CPU frequency range: 80 - 240 MHz

[POWER] Light sleep timing configured:
  Upload cycle: 900000 ms (15.0 min)
  Start buffer: 135000 ms (2.2 min)
  Sleep window: 630000 ms (10.5 min)
  End buffer: 135000 ms (2.2 min)

[SLEEP] Entering sleep window (remaining: 630000 ms)
[WAKE] Sleep window completed, resuming operations
```

9. Configuration Options

9.1 Adjustable Parameters

Parameter	Location	Default	Adjustment Range	Impact
-----------	----------	---------	------------------	--------

Parameter	Location	Default	Adjustment Range	Impact
Sleep Percentage	<code>main.cpp:82</code>	70%	60-80%	Balance power vs stability
Start Buffer	<code>main.cpp:83</code>	15%	10-20%	WiFi stability time
End Buffer	<code>main.cpp:84</code>	15%	10-20%	Upload completion time
Min Sleep Duration	<code>main.cpp:112</code>	5000 ms	3000-10000 ms	Prevent short sleeps
Max CPU Frequency	<code>main.cpp:289</code>	240 MHz	160-240 MHz	Active performance
Min CPU Frequency	<code>main.cpp:290</code>	80 MHz	40-80 MHz	Idle power

9.2 Configuration Profiles

Profile	Sleep %	Start %	End %	Use Case
Conservative	60	20	20	Unstable WiFi, frequent uploads
Balanced (default)	70	15	15	Normal operation
Aggressive	80	10	10	Stable WiFi, maximum savings

10. Theory of Operation

10.1 FreeRTOS Tickless IDLE

Normal FreeRTOS Operation:

- Periodic tick interrupt: Every 10ms (100Hz tick rate)
- CPU wakes on every tick even if no tasks ready
- Prevents long sleep periods

Tickless IDLE Operation:

- Monitors all task states
- When all tasks blocked: Calculates next wakeup time
- Disables periodic tick interrupt
- Configures one-shot timer for wakeup
- Allows CPU to sleep for calculated duration

Sleep Decision Logic:

- 1. Check all tasks → All blocked?
NO → Continue normal scheduling
YES → Calculate next unblock time
- 2. Next unblock time > minimum threshold (30ms)?
NO → Stay awake (overhead not worth it)
YES → Enter tickless idle
- 3. Stop periodic tick interrupt
- 4. Configure wakeup timer
- 5. Call esp_pm for light sleep
- 6. Sleep until timer or task event

10.2 Power Management Component

Dynamic Frequency Scaling (DFS):

- Scales CPU frequency based on load
- Reduces frequency during light operations
- Increases frequency for heavy operations
- Automatic voltage scaling with frequency

Light Sleep Coordination:

- Monitors system state (WiFi, Bluetooth, peripherals)
- Ensures safe sleep conditions
- Manages peripheral power states
- Coordinates wakeup sources

WiFi Modem Sleep:

- Maintains WiFi association during light sleep
- Reduces WiFi power from ~80mA to ~10mA
- Tracks DTIM beacons for synchronization
- Automatic data buffering by access point

11. Comparison with Alternatives

11.1 Sleep Mode Options

Mode	CPU State	RAM	WiFi	Wake Time	Typical Current
------	-----------	-----	------	-----------	-----------------

Mode	CPU State	RAM	WiFi	Wake Time	Typical Current
Active	Running	Active	Active	N/A	150-240 mA
Modem Sleep	Running	Active	Power save	N/A	15-60 mA
Light Sleep	Suspended	Retained	Modem sleep	1-3 ms	0.8-1.1 mA
Deep Sleep	Off	Lost (RTC only)	Off	100-300 ms	10-150 µA

11.2 Why Light Sleep for This Application

Requirement	Light Sleep	Deep Sleep	Justification
Context preservation	✓ RAM retained	✗ RAM lost	Need task states, buffers
Fast wake	✓ 1-3 ms	✗ 100-300 ms	Quick response to events
WiFi maintenance	✓ Connection kept	✗ Reconnect needed	Avoid reconnect overhead
Implementation	✓ FreeRTOS native	✗ Manual restart	Cleaner code integration
Power savings	✓ Good (0.8-1.1 mA)	✓ Better (10-150 µA)	Sufficient for use case

12. Expected Validation and Testing

12.1 Power Measurement Setup

Equipment:

- USB power monitor or digital multimeter
- ESP32 powered via monitored power supply
- Stable WiFi connection

Measurement Points:

1. Measure current over complete 15-minute cycle
2. Verify active period current (100-150 mA range)
3. Verify sleep period current (< 5 mA)
4. Calculate weighted average

12.2 Expected Results

Metric	Expected Value	Tolerance	Status Indicator
--------	----------------	-----------	------------------

Metric	Expected Value	Tolerance	Status Indicator
Active current	100-150 mA	±30 mA	Serial log: "[ACTIVE]"
Sleep current	0.8-3 mA	±2 mA	Serial log: "[SLEEP]"
Wake time	< 5 ms	±5 ms	Measured via GPIO toggle
WiFi reconnections	0 per cycle	0	Serial log: "Connected"

We're checking WiFi status constantly (every 2 seconds in CloudConnect task), but we only check once before sleep in the main_task. The modem sleep mode is designed to maintain the connection without needing explicit checks after wake.

13. Troubleshooting

13.1 Common Issues

Issue	Symptom	Likely Cause	Solution
Sleep not activating	Current stays high	Missing 32kHz crystal	Check GPIO32/33 connections
Build errors	PM functions undefined	Wrong build environment	Use <code>esp32dev hw</code> not <code>esp32dev_sim</code>
WiFi disconnects	Reconnect messages	Weak signal or long sleep	Reduce sleep % or improve signal
High sleep current	> 10 mA during sleep	Peripheral not disabled	Check for active LED/sensors
Inaccurate timing	Sleep duration off	Crystal calibration	Check <code>RTC_CLK_CAL_CYCLES</code> flag

13.2 Debug Serial Output

Enable detailed logging:

```
// In main.cpp, add position logging
Serial.printf("[DEBUG] Cycle position: %u / %u ms\n",
              cyclePosition, UPLOAD_PERIOD_MS);
```

Monitor for:

- Power management configuration success/failure

- Sleep window entry/exit timestamps
 - WiFi status before/after sleep
 - Upload success after wake
-

14. Summary

14.1 Key Features

Feature	Implementation	Benefit
Automatic Sleep	FreeRTOS Tickless IDLE	No manual timing management
RTOS Integration	Native scheduler support	Safe task coordination
WiFi Preservation	Modem sleep mode	No reconnection overhead
Centered Window	15-70-15 timing split	WiFi stability + max savings
Dual Mode	Simulation vs production	Same code, different builds

14.2 Performance Summary

Power Reduction: 70-80% average (based on 70% sleep duty cycle)

Wake Latency: < 5 ms (light sleep characteristics)

WiFi Stability: Maintained through sleep (modem sleep mode)

Implementation Complexity: Low (configuration-based)

14.3 Quick Reference

Build for Testing:

```
pio run -e esp32dev_sim -t upload
```

Build for Production:

```
pio run -e esp32dev_hw -t upload
```

Key Configuration:

- Upload interval: 900 seconds (15 minutes)
- Sleep window: 630 seconds (70%)

- Active time: 270 seconds (30%)
 - Expected average: 30-50 mA
-

Power Optimization Method: ESP-IDF PM + FreeRTOS Tickless IDLE